Manual técnico

1. Descripción general

REDXela es un simulador de centro de distribución que administra clientes, pedidos, recepción de mercancía, inventario y almacenamiento físico. Procesa pedidos mediante trabajadores concurrentes y permite estudiar el comportamiento de los recursos compartidos, las políticas de planificación y la recuperación ante bloqueos mutuos.

El backend ejecuta la lógica operativa de forma independiente del navegador. El frontend ofrece los controles y muestra estados, estadísticas y eventos en tiempo real. No se utiliza una base de datos: la información se conserva únicamente durante la ejecución del servidor.

2. Arquitectura y tecnologías

La aplicación está organizada como un backend monolítico modular con API REST y una interfaz web SPA independiente.

Componente	Tecnología principal	Responsabilidad
Backend	Java 21 · Spring Boot 4.1.2	Reglas del negocio, planificación y ejecución concurrente
Frontend	Vue 3 · Vite 8 · Vue Router	Interfaz, navegación y consulta de resultados
Estilos	Tailwind CSS 4	Diseño adaptable y componentes visuales
Comunicación	REST · SSE	Comandos HTTP y actualizaciones de estado
Concurrencia	Java Concurrency API	Hilos, semáforos, locks y colas de mensajes
Persistencia	Memoria del proceso	Clientes, pedidos, inventario y métricas

Estructura del proyecto

```
redxela-api/      Backend Spring Boot y pruebas
redxela-client/   Aplicación web Vue 3
redxela-api/src/main/java/com/redxela/api/features/   Módulos de dominio
redxela-client/src/features/                Vistas y servicios
```

Diseño general. Vue consume endpoints REST mediante Fetch y escucha snapshots SSE; las operaciones críticas y el cálculo de prioridades se realizan exclusivamente en Java.

3. Requisitos y puesta en marcha

Para ejecución local se requiere Java 21 LTS, Node.js compatible con el proyecto (recomendado 24.22.1), npm y acceso a las carpetas del repositorio. El backend incluye Maven Wrapper, por lo que no es necesario instalar Maven globalmente.

Iniciar el backend

```
cd redxela-api
./mvnw spring-boot:run
```

Iniciar el frontend (segunda terminal)

```
cd redxela-client
npm ci
npm run dev
```

Servicio	Dirección local
Backend	http://localhost:8080
Frontend	http://localhost:5173
Dashboard	http://localhost:5173/dashboard
Conectividad REST	GET http://localhost:8080/api/health

Arranque. El inventario inicia vacío y el motor de pedidos detenido. Recepción inicia con Spring Boot; el generador automático y el motor se activan por separado desde la interfaz.

4. Componentes de la lógica de negocio

El código del backend está organizado por funcionalidad bajo el paquete com.redxela.api.features. Estas son las clases centrales para comprender y mantener el sistema.

Clase / servicio	Responsabilidad
OrderService · OrderIntakeService	Registro, estados de pedidos y admisión con reserva de stock
OrderGenerator · OrderScheduler	Llegadas automáticas, selección de pedidos y políticas
ProcessingEngine	Pool de cuatro trabajadores y ejecución por etapas
ResourceManager	Control de recursos y asignación sincronizada
InventoryService · PhysicalStorage	Existencias, reservas, lotes y ubicaciones
ReceptionService	Mensajería entre recepción e inventario
DeadlockService	Creación, detección y recuperación de espera circular
OperationalSnapshotService · SimulationEvents	Métricas y snapshots enviados por SSE

5. Flujo de pedidos e inventario

Los pedidos se crean manualmente o por el generador automático. Su procesamiento depende de contar con todas las existencias requeridas y con los recursos necesarios para cada etapa.

Etapa	Comportamiento
Ingreso	Se crea el pedido con cliente, nivel de servicio y productos.
Reserva	Inventario reserva todas las líneas o mantiene el pedido esperando existencias.
Ejecución	Un trabajador realiza recogida, calidad (si aplica), empaque, escaneo y carga.
Despacho	Al finalizar se consume la reserva y el pedido pasa a completado.
Cancelación	Se liberan reservas no consumidas y se reevalúan pedidos pendientes.

La reserva es atómica: un pedido no recibe únicamente una parte de los productos solicitados. Las existencias físicas incluyen las unidades reservadas hasta su consumo; así se evita descontar dos veces o producir cantidades negativas.

Recepción y almacenamiento

Recepción y el consumidor de Inventario trabajan de forma independiente y se comunican mediante BlockingQueue. Una llegada aceptada se escanea, se registra como mensaje y finalmente actualiza existencias y ubicaciones. El identificador UUID permite evitar duplicar entradas cuando se reintenta un mensaje.

El almacén cuenta con 3 pasillos, 2 niveles por pasillo y 5 posiciones por nivel: 30 ubicaciones de 20 unidades, para un máximo de 600. Los lotes pueden fragmentarse entre ubicaciones diferentes conservando su identificador.

6. Ejecución concurrente y recursos

ProcessingEngine utiliza cuatro trabajadores Java que pueden atender pedidos en paralelo. ResourceManager limita la ocupación de cada tipo de recurso y registra las unidades asignadas y las operaciones en espera.

Recurso compartido	Capacidad
Estaciones de empaque	6
Áreas de carga	2
Encargados de bodega	10
Montacargas	3
Control de calidad	4
Escáner de recepción y despacho	1

Sincronización. Semaphore limita el uso simultáneo; ReentrantLock y Condition coordinan esperas y liberaciones. En el flujo normal, cada operación obtiene el conjunto necesario o espera sin retener recursos parciales.

7. Planificación, reloj y generación

El planificador selecciona pedidos con inventario reservado. Los pedidos esperando existencias no forman parte de la cola elegible. Se puede cambiar la política activa desde la interfaz sin eliminar pedidos pendientes.

Política	Criterio
Prioridad estricta	Primero el nivel de servicio de mayor prioridad.
Aging	Eleva la prioridad efectiva de los pedidos conforme acumulan espera; un nivel cada 300 segundos simulados.
SJF	Atiende primero los pedidos con menor número total de unidades.

El laboratorio de comparación ejecuta el mismo conjunto de entradas para las tres políticas, con una semilla común. Los resultados son estimaciones independientes: no completan pedidos reales ni consumen inventario.

Tiempo simulado y generador

El reloj opera inicialmente a 60 segundos simulados por cada segundo real; el trabajo base es de 10 segundos simulados por unidad. Estos parámetros están en redxela-api/src/main/resources/application.yaml.

OrderGenerator produce pedidos automáticos mediante un ejecutor programado. La semilla inicializa java.util.Random y determina la selección de productos, cantidades y variación del intervalo entre llegadas. El intervalo configurado varía del 50 % al 150 %; se puede iniciar, pausar, reanudar y detener sin cerrar el motor.

8. Deadlock y recuperación manual

El sistema incluye un escenario controlado de bloqueo mutuo con dos hilos. La operación A retiene SCANNER-01 y solicita FORKLIFT-01; la operación B retiene FORKLIFT-01 y solicita SCANNER-01. Así se produce una espera circular observable sobre recursos reales.

Operación A	Operación B
Retiene: SCANNER-01	Retiene: FORKLIFT-01
Espera: FORKLIFT-01	Espera: SCANNER-01

El operador selecciona cuál continúa. La operación postergada se interrumpe cooperativamente, libera su concesión y vuelve a espera; la elegida finaliza y luego se reintenta la otra. No se utiliza Thread.stop() ni se fuerzan liberaciones desde la petición HTTP.

Alcance. Este experimento permite deliberadamente la retención parcial de recursos. La ejecución normal de pedidos mantiene una política de adquisición que evita esa condición.

9. API, monitoreo y métricas

Spring MVC publica endpoints REST para consultas y comandos. El frontend utiliza Fetch para operaciones puntuales y EventSource para recibir snapshots SSE de forma periódica.

Endpoint principal	Uso
GET /api/health	Estado del backend
/api/customers · /api/orders	Clientes y registro de pedidos
/api/reception/arrivals	Registro asíncrono de mercancía
/api/generator · /api/simulation	Control del generador y motor
/api/scheduling · /api/resources	Políticas y disponibilidad
/api/inventory · /api/storage	Existencias, reservas y ubicaciones
/api/deadlock	Provocación, consulta y resolución del conflicto
GET /api/operations/events	Snapshots SSE de la operación

Los snapshots muestran pedidos recibidos, en proceso, completados y en espera; disponibilidad de recursos, ocupación de almacén, tiempos de espera y procesamiento, y actividad reciente. El navegador se reconecta tras una interrupción y recupera el estado actual mediante nuevas consultas.

10. Verificación y mantenimiento

Las comprobaciones esenciales consisten en ejecutar las pruebas del backend, compilar ambos proyectos y revisar el recorrido de registro de pedido, entrada de stock, procesamiento y despacho. También debe verificarse la liberación de recursos tras un conflicto.

Pruebas del backend

```
cd redxela-api
./mvnw test
./mvnw -DskipTests clean package
```

Comprobación del frontend

```
cd redxela-client
npm run lint
npm run build
```

Consideraciones de operación

Toda la información se conserva en memoria y se pierde al reiniciar Spring Boot. No existe autenticación de usuarios, base de datos ni despliegue productivo preconfigurado. Los tiempos representan una simulación y el almacén trata cada unidad con la misma capacidad física, independientemente de su tamaño o peso reales.

Para mantener la consistencia, cualquier ampliación debe preservar las reservas completas, la liberación de permisos por su hilo propietario, la idempotencia de las llegadas y la independencia entre ejecución del motor y navegador.